

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

12 de septiembre de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 3: Comunicación y coordinación

Objetivos:

1. Pasos de Mensaje: MPI, Mensajes Punto a Punto, MPI, Comunicación Colectiva,
2. Blocking vs non-blocking
3. Tipos MPI
4. Memoria Compartida: OMP, Constructores y cláusulas
5. Programacion Híbrida

Tipos MPI

MPI debe funcionar en sistemas potencialmente heterogéneos, donde los procesos pueden ejecutarse sobre arquitecturas diferentes. Por esta razón, MPI no describe los mensajes simplemente mediante cantidades de bytes, sino mediante tipos de datos

MPI Datatype	C datatype
MPI_CHAR	signed char
MPI_SHORT	signed short int
MPI_INT	signed int
MPI_LONG	signed long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	
MPI_PACKED	

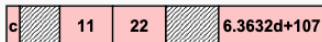
Figura 1: *Tipos básicos MPI*

Tipos derivados MPI

Son tipos construidos en base a los tipos básicos MPI (e.g. MPI_INT, MPI_FLOAT, ...)

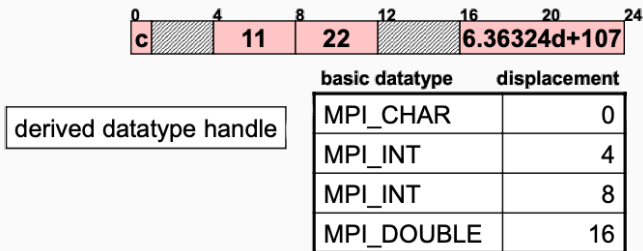
Muchas veces se necesita:

- enviar data no-continua de un solo tipo (e.g. una submatriz) o distintos tipos
- enviar data continua de distintos tipos (e.g., un contador entero, seguido de doubles)



Tipos derivados MPI

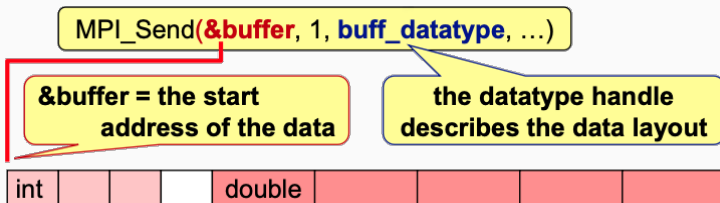
Tipos derivados MPI son punteros a una lista de entradas confirmadas por tipos básicos y desplazamientos. Estos describen la estructura de la data en memoria, e.g. de structs, subarrays



Tipos derivados MPI

MPI brinda herramientas para:

- no realizar múltiples llamadas MPI para enviar cada elemento (múltiples latencias)
- No copiar data en arrays contiguos (evitar comunicación)



Pasos de implementacion de un Tipo Derivado:

1. **Construcción:** Llamar a una función constructora (ej. `MPI_Type_vector`).
2. **Confirmación (Commit):** Llamar a `MPI_Type_commit` antes de usarlo en comunicaciones.
3. **Uso:** Emplearlo en funciones como `MPI_Send` o `MPI_Bcast`.
4. **Liberación:** Llamar a `MPI_Type_free` para liberar recursos de memoria una vez que ya no se necesite.

MPI_Type_contiguous

int MPI_Type_contiguous (int count, MPI_Datatype old_type,
MPI_Datatype * new_type);

```
count =4;  
MPI_Type_contiguous(count , MPI_FLOAT,&rowtype);  
MPI_Send(&a[2][0] ,1 ,rowtype ,dest ,tag ,comm);
```

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0	10.0	11.0	12.0
13.0	14.0	15.0	16.0

9.0	10.0	11.0	12.0
-----	------	------	------

Figura 2: *Se envía un elemento de tipo rowtype*

La ventaja se ve cuando se envía mas de una fila, teniendo cada fila un nombre lógico (e.g. 'rowtype')

MPI_Type_contiguous (ejemplo)

$$P = \begin{bmatrix} x_0 & y_0 & z_0 \\ x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \end{bmatrix} = \begin{bmatrix} P_0 \\ P_1 \\ P_2 \end{bmatrix}, \quad P_i = (x_i, y_i, z_i)$$

$$\underbrace{x_i \quad y_i \quad z_i}_{\text{punto3D}}$$

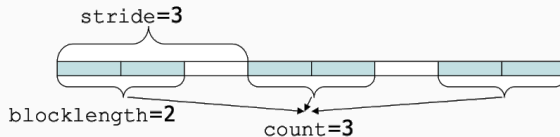
`punto3D = MPI_Type_contiguous(3, MPI_DOUBLE)`

```
MPI_Datatype punto3D;  
MPI_Type_contiguous(3, MPI_DOUBLE, &punto3D);  
MPI_Type_commit(&punto3D);  
MPI_Send(&P[i][0], 1, punto3D, destino, tag, MPI_COMM_WORLD);  
MPI_Type_free(&punto3D);
```

MPI_Type_vector

MPI_Type_vector (int count, int blocklength, int stride, MPI_Datatype datatype, MPI_Datatype *newtype);

count: cantidad de bloques de longitud **blocklength** (elementos del mismo tipo). De inicio a inicio de cada bloque, hay una distancia **stride** de elementos del mismo tipo.



MPI_Type_vector

```
count = 4; blocklength = 1; stride = 4;  
MPI_Type_vector(count, blocklength, stride, MPI_FLOAT,  
                &columntype);
```

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0	10.0	11.0	12.0
13.0	14.0	15.0	16.0

a[4][4]

```
MPI_Send(&a[0][1], 1, columntype, dest, tag, comm);
```

2.0	6.0	10.0	14.0
-----	-----	------	------

1 element of
columntype

MPI_Type_indexed (int count, int blocklengths[], int displacements[], MPI_Datatype datatype, MPI_Datatype *newtype);

count: cantidad de bloques

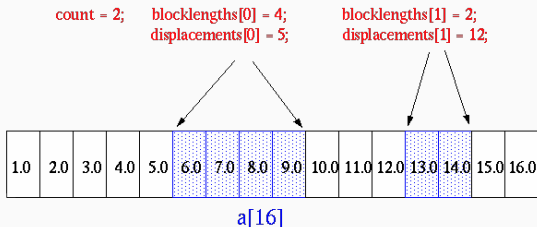
blocklengths: array conteniendo la longitud de cada bloque

displacements: array conteniendo las distancias de cada bloque desde el inicio al inicio de cada bloque

datatype: el tipo básico MPI

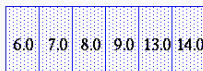
newtype: variable que almacena el tipo generado

MPI_Type_indexed



```
MPI_Type_indexed(count, blocklengths, displacements, MPI_FLOAT, &indextype);
```

```
MPI_Send(&a, 1, indextype, dest, tag, comm);
```



1 element of
indextype

Son una forma simple y eficiente de comunicación de tipos combinados y no-continuos en un solo mensaje. Ya que el tipo de data MPI especifica claramente el espacio en memoria de cada una de las componentes para ser enviados o recibidos.

Consisten en una secuencia de tipos MPI y desplazamientos en memoria de los mismos desde el inicio del tipo derivado.

Por ejemplo, un tipo derivado podría consistir en dos doubles y un int, almacenados en memoria como sigue:

a, direccion (24)

b, direccion (40)

n, direccion (48)

Y el tipo derivado sería :

{(MPI_DOUBLE,0), (MPI_DOUBLE,16),(MPI_INT,24)}

MPI_Type_create_struct

Para construir el tipo derivado usamos

```
int MPI_Type_create_struct ( int count, int array_of_blocklengths[],  
MPI_Aint array_of_displacements[], MPI_Datatype array_of_types[],  
MPI_Datatype* new_type_p );
```

En el ejemplo,

count=3

array_of_blocklengths[3]={1,1,1}

array_of_displacements[]={0,16,24}

Para encontrarlos, podemos usar `int MPI_Get_address (void* location_p,
MPI_Aint* address_p);`

que retorna la dirección `location_p` (`MPI_Aint` es de tipo entero que almacena direcciones)

MPI_Type_create_struct

Entonces, podemos usar el siguiente código

```
MPI_Aint a_addr, b_addr, n_addr;
MPI_Get_address(&a, &a_addr);
array_of_displacements[0] = 0;
MPI_Get_address(&b, &b_addr);
array_of_displacements[1] = b_addr-a_addr;
MPI_Get_address(&n, &n_addr);
array_of_displacements[2] = n_addr-a_addr;
MPI_Datatype array_of_types[3] =
    { MPI_DOUBLE, MPI_DOUBLE, MPI_INT};
MPI_Datatype input_mpi;
MPI_Type_create_struct (3, array_of_blocklengths,
    array_of_displacements, array_of_types, &input_mpi);
int MPI_Type_commit(MPI_Datatype* new_mpi );
MPI_Bcast(&a, 1, input_mpi, 0, comm);
```

MPI_Type_create_struct

```
void Build_mpi_type(
    double* a_p          /* in */,
    double* b_p          /* in */,
    int* n_p             /* in */,
    MPI_Datatype* input_mpi_t_p /* out */) {

    int array_of_blocklengths[3] = {1, 1, 1};
    MPI_Datatype array_of_types[3] = {MPI_DOUBLE, MPI_DOUBLE, MPI_INT};
    MPI_Aint a_addr, b_addr, n_addr;
    MPI_Aint array_of_displacements[3] = {0};

    MPI_Get_address(a_p, &a_addr);
    MPI_Get_address(b_p, &b_addr);
    MPI_Get_address(n_p, &n_addr);
    array_of_displacements[1] = b_addr - a_addr;
    array_of_displacements[2] = n_addr - a_addr;
    MPI_Type_create_struct(3, array_of_blocklengths,
        array_of_displacements, array_of_types,
        input_mpi_t_p);
    MPI_Type_commit(input_mpi_t_p);
} /* Build_mpi_type */

void Get_input(int my_rank, int comm_sz, double* a_p, double* b_p,
    int* n_p) {
    MPI_Datatype input_mpi_t;

    Build_mpi_type(a_p, b_p, n_p, &input_mpi_t);

    if (my_rank == 0) {
        printf("Enter a, b, and n\n");
        scanf("%lf %lf %d", a_p, b_p, n_p);
    }
    MPI_Bcast(a_p, 1, input_mpi_t, 0, MPI_COMM_WORLD);

    MPI_Type_free(&input_mpi_t);
} /* Get_input */
```

Una alternativa a construir tipos estructurados formales es empaquetar variables secuencialmente en un arreglo de bytes (`MPI_PACKED`) antes de enviarlo.

- **Ventaja:** Es extremadamente dinámico. Ideal cuando la estructura del mensaje varía en tiempo de ejecución o es una comunicación de un solo uso.
- **Desventaja:** Incurrir en un costo extra por copiar explícitamente los datos de las variables al *buffer* de empaquetado por parte de la CPU (overhead de memoria).

El `MPI_Type_create_struct` es preferible cuando la comunicación del mismo tipo de estructura se realiza de forma recurrente dentro de un bucle.

Operación de envío y recibo inmediato de mensajes.

```
MPI_Sendrecv(&sendbuf,sendcount,sendtype,dest,sendtag,  
&recvbuf,recvcount,recvtype,source,recvtag,comm,status)
```

- **sendbuf, recvbuf**: puntero a la variable que ocupa la data que va a ser enviada o recibida (nombre de la variable)
- **send/recv count**: número de elementos de la data que serán enviados/recibidos
- **send/recv datatype**: tipo de variable MPI enviada o recibida
- **dest, source**: proceso destino/origen del mensaje (rank)
- **send/recv tag**: identificador del proceso que envía/recibe.
- **comm**: comunicador del grupo de procesos activo
- **status**: estado de la comunicación

-  <https://www.mpich.org/documentation/guides/>
-  William Gropp, Ewing Lusk and Anthony Skjellum: *Using MPI: Portable Parallel Programming With the Message-Passing Interface*. MIT Press, 3rd edition, Nov. 2014 (336 pages, ISBN 9780262527392)
-  William Gropp, Torsten Hoefler, Rajeev Thakur and Ewing Lusk: *Using Advanced MPI: Modern Features of the Message-Passing Interface*. MIT Press, Nov. 2014 (392 pages, ISBN 9780262527637)
-  Neil MacDonald, Elspeth Minty, Joel Malard, Tim Harding, Simon Brown, Mario Antonioletti: *Parallel Programming with MPI. Historical MPI course notes from EPCC*.
http://www.archer.ac.uk/training/course-material/2014/10/MPI_UCL/Notes/MPP-notes.pdf



Peter S. Pacheco. *An Introduction to Parallel Programming*. 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5. Ch 3



Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656. Ch 4

